

Cross-Model Generalization in AI Text Detection Comparing Feature-Based and Neural Approaches

Melissa Amenuveve | Florida State University | CAP 5638 Pattern Recognition | Spring 2026

GitHub Repository: <https://github.com/Mamenuveve2201/Cross-Model-Generalization-in-AI-Text-Detection-Comparing-Feature-Based-and-Neural-Approaches>

Live Web App: <https://huggingface.co/spaces/amenuvevemelissa/ai-detector>

Introduction

It is getting harder to tell whether something was written by a person or a machine. GPT-4o, Claude, Gemini, and LLaMA have all reached a point where their writing passes for human in most situations. Detection tools exist, but they are not keeping up with how fast these models have improved.

What makes detection difficult is not just that AI writing has improved. It is that detectors do not travel well. A detector that works on GPT-2 output tends to fall apart when you show it something from Gemini. Each model has its own patterns, and detectors trained on one model's patterns do not automatically pick up another model's patterns. That is the specific problem this project set out to study.

Xia et al. looked at this and found that some linguistic features show up in AI writing across multiple models, not just one. Passive voice, tense distributions, pronoun use. Their work explained why detectors struggle to generalize. What it did not do was test whether those features could be used to build a detector that generalizes. That is the gap we tried to fill.

The project contributes four things. A dataset of 5,981 samples drawn from five different LLMs and real student essays, with each AI essay matched to a human essay by topic and length. A leave-one-model-out evaluation setup built specifically to test generalization. A set of 41 linguistic features extracted from each essay and used to train classical ML classifiers. And a result we did not fully expect: those classifiers outperformed fine-tuned neural detectors by over 81 F1 points when tested on models they had never seen.

Related Work

AI text detection has gone in two directions. One side built detectors around hand-crafted features like perplexity and burstiness. The other side went neural. GLTR came from the first camp. Gehrmann et al. noticed that AI text tends to cluster near the top of a model's probability distribution and built a visualization tool around that. Mitchell et al. went a different route with DetectGPT, flagging AI text through perturbation-based scoring without training a classifier at all.

Neural detectors came along later. OpenAI put out a RoBERTa detector trained on GPT-2 output. Hello-SimpleAI released one fine-tuned on ChatGPT text. Both worked well on what they were trained on. Neither was seriously tested on output from a model they had never seen.

Xia et al. is the closest thing to what we did. They looked at which linguistic features explain why detectors break down on unfamiliar models. Word length, type-token ratio, sentence length kept coming up. These patterns appeared across multiple models, not just one. But their work stopped at explanation. No one had taken those same features and used them to build a detector. That is where this project picks up.

Our human essays come from the PERSUADE corpus, a student writing dataset from Kaggle. It gave us a solid, consistent human baseline to work with. Single-model detection has been done before, the DAIGT competition being one example. What has not been done is putting classical ML, neural detectors, and zero-shot methods through the exact same leave-one-model-out setup at once.

Methodology

Dataset

Our dataset contains 5,981 samples: 1,000 human essays from the PERSUADE 2.0 corpus and AI-generated essays from five LLMs. The target was 6,000 samples (1,000 per source), but 19 samples were removed due to API rate limit failures, empty responses, and truncated outputs. Table 1 summarizes the final counts.

Source	Model/Corpus	Target	Actual	API	Output File
Human	PERSUADE 2.0 Corpus	1,000	1,000	—	persuade_samples.csv
AI	GPT-4o	1,000	993	openai	gpt4o_samples.csv
AI	Claude Sonnet 4	1,000	999	anthropic	claude_samples.csv
AI	Gemini 2.5 Flash	1,000	993	google. generativeai	gemini_samples.csv
AI	LLaMA 3.3 70B	1,000	1,000	Groq API	llama_samples.csv
AI	Qwen3 32B (Alibaba)	1,000	996	Groq API	qwen_samples.csv
Total	6 sources	6,000	5,981	—	combined_dataset.csv

We used the first 1,000 rows of the PERSUADE corpus as our human baseline. Every AI essay was generated to match the topic and approximate length of the human essay at the same index. The idea was simple: if an AI essay and a human essay are about the same topic and roughly the same length, any difference the classifier picks up had to come from how they are written, not what they are about. A detector that just learns to recognize essay length or topic will not hold up across models. Matching on those variables forces the classifier to focus on style.

Essay Generation

Every model got the same prompt:

"Write an argumentative essay about {topic} that is approximately {essay_length} characters long."

No examples. No system prompt. Just that one instruction. The topic came from a prior step where GPT-4o extracted it from each PERSUADE essay and stored it in the extracted_topic column. The target length was just the character count of the matching human essay.

Each script went through the first 1,000 rows of persuade_with_topics.csv, called the API, and recorded what came back. A 0.5 second pause between calls kept us from hitting rate limits. Anything that failed got logged with a GENERATION_FAILED label and left out of the analysis.

The five models we used were GPT-4o through the OpenAI API, Claude Sonnet 4 through the Anthropic API, Gemini 2.5 Flash through Google's generativeai library, and LLaMA 3.3 70B Versatile and Qwen3 32B both through the Groq API. Exact model strings were gpt-4o, claude-sonnet-4-20250514, models/gemini-2.5-flash, llama-3.3-70b-versatile, and qwen/qwen3-32b. Generation counts came out to 993, 999, 993, 1,000, and 996 respectively.

Every output record has the same six fields: essay_id, source_id, topic, target_length, generated_text, and actual_length. All five sources were merged into combined_dataset.csv with a label column where 0 means human and 1 means AI, plus a model column identifying which generator produced each sample.

Feature Extraction

We extracted 41 features per essay organized into four categories. We started with 57 but cut 16 of them. The ones we dropped all needed an external language model to compute and that made the pipeline too slow to work with practically.

What was left breaks into four groups.

The first is stylometric. Twelve features that look at word patterns. How long words are, how rich the vocabulary is, how many words only show up once.

The second is syntactic. Ten features based on parts of speech. How much of the essay is nouns, verbs, pronouns, and so on. Human writers and AI models land in pretty different places on these.

The third group came from Xia et al. Ten semantic features covering voice, tense, and pronoun person frequencies. We kept them in to find out whether features that explain detection failures can also predict them.

The fourth group had nine statistical features. Entropy, burstiness, repetition patterns, and sentences start diversity.

Evaluation Protocol

We used leave-one-model-out cross-validation with 5 folds. In each fold one AI model gets held out as the test set. The training set is everything else: human essays plus essays from the four remaining AI models. Human samples are split evenly, 200 go into each test fold and 800 go into training. The scaler is fitted on training data only so nothing leaks.

Each fold had a different test size depending on how many essays that model produced. GPT-4o was 1,193 samples, Claude was 1,199, Gemini was 1,193, LLaMA was 1,200, and Qwen was 1,196. We tracked four metrics: F1 score with AI as the positive class, accuracy, AUC-ROC, and false positive rate. False positive rate here means how often a human essay got flagged as AI.

Models

Three classical ML models were trained on the 41 features. Logistic Regression with L2 penalty, C=1.0, and max_iter=1000. Random Forest with 100 estimators and max_depth=10. XGBoost with learning_rate=0.1, max_depth=6, and 100 estimators. All three outputs feature importance scores which we look at later.

Two pre-trained RoBERTa models served as neural baselines. We did not fine-tune either of them. The first is RoBERTa-base-openai-detector. It was trained on GPT-2 output by OpenAI. The second is Hello-SimpleAI's chatgpt-detector-roberta, trained on ChatGPT text. None of our five models were in their training data so they are a genuine test of out-of-the-box generalization.

We originally planned a hybrid ensemble on top of everything, but we skipped that once we saw that the classical models were hitting 99% F1 on their own, stacking an 18% F1 neural model on top of that made no sense. We flagged it as future work and moved on.

Experimental Results

Overall Performance

Table 2 presents mean performance across all 5 LOMO-CV folds.

Classifier	Mean F1	Mean Acc.	Mean AUC	Mean FPR	Type
Logistic Regression	99.94%	99.90%	1.0000	0.00%	Classical
Random Forest	99.61%	99.32%	0.9999	0.00%	Classical
XGBoost	99.63%	99.40%	0.9999	0.20%	Classical
RoBERTa-OpenAI	18.37%	27.84%	0.5708	4.40%	Neural
ChatGPT Detector	18.27%	27.47%	0.4169	11.90%	Neural

The gap is hard to ignore. All three classical classifiers came in above 99% mean F1. Both neural baselines landed around 18%. That is over 81 F1 points between them, and it was not close.

The false positive rate told an even bigger story. Logistic Regression and Random Forest had zero false positives. Not a single human essay got flagged. The ChatGPT Detector got it wrong on roughly 1 in 8 human essays. If you are using that tool to catch cheating, you are accusing the wrong person almost 12% of the time.

Per-Fold Results

Table 3 shows the full breakdown across all 25-fold-classifier combinations.

Held-Out Model	LR F1	RF F1	XGB F1	RoBERTa F1	ChatGPT-Det F1	Note
GPT-4o (n=1,193)	1.0000	1.0000	1.0000	0.1256	0.0628	—
Claude 3.5 (n=1,199)	1.0000	1.0000	1.0000	0.0000	0.0000	AUC<0.5
Gemini 1.5 (n=1,193)	1.0000	1.0000	0.9990	0.0000	0.0000	AUC<0.5
LLaMA 3.1 (n=1,200)	0.9970	0.9822	0.9837	0.7693	0.8216	Hardest ⚠
Qwen (n=1,196)	1.0000	0.9985	0.9990	0.0236	0.0292	—

LLaMA was consistently the hardest model to catch. Logistic Regression still hit 99.70% F1 on it, but Random Forest dropped to 98.22% and XGBoost to 98.37%. Those were the lowest numbers in the whole experiment. The neural baselines did better on LLaMA than on any other model, 76.93% and 82.16% F1. Our guess is that LLaMA's writing patterns overlap more with the older GPT-era text those detectors were trained on.

The neural baselines completely fell apart on Claude, Gemini, and Qwen. F1 of 0.00% on all three. AUC below 0.5, which means they were doing worse than a coin flip. That is not a small failure. It means those detectors learned what one specific model looks like, not what AI writing looks like in general.

Only 4 samples out of 5,981 fooled all three classical classifiers at the same time. That is a 0.07% error rate. All four were LLaMA essays that got misclassified as human. LLaMA just writes in a way that looks more human than the other models we tested.

Feature Importance

Table 4 shows the top 10 features by average importance across Random Forest and XGBoost folds.

Rank	Feature	Category	RF Imp.	XGB Imp.	Interpretation
1	syllables_per_word	Stylometric	0.1799	0.8325	AI uses more polysyllabic words
2	avg_word_len	Stylometric	0.1303	0.0348	AI uses longer words on average

3	long_word_ratio	Stylometric	0.1290	0.0263	AI uses more words >7 characters
4	hapax_ratio	Stylometric	0.1229	0.0128	Words used exactly once / vocab size
5	lexical_density	Stylometric	0.1099	0.0084	Content words as % of total words
6	word_length_variance	Stylometric	0.0754	0.0028	Variance in word lengths
7	type_token_ratio	Stylometric	0.0546	0.0043	Vocabulary diversity measure
8	repetition_rate	Statistical	0.0419	0.0026	How often word pairs repeat
9	pos_pronoun_ratio	Syntactic	0.0355	0.0028	Humans use more pronouns
10	short_word_ratio	Stylometric	0.0341	0.0001	Humans use more short words

syllables_per_word ended up carrying over half the weight across both Random Forest and XGBoost combined. That one feature alone outweighed all 40 others put together. AI writing consistently uses harder and longer words and that signal turned out to be incredibly stable across every model we tested.

The rest of the top five followed the same pattern. avg_word_len, long_word_ratio, hapax_ratio, and lexical_density were all vocabulary complexity measures. Together the five of them covered over 81% of total importance.

The features Xia et al. pointed to a different story. Passive voice, tense frequencies, and pronoun frequencies all ranked somewhere between 14th and 40th place. They barely registered in terms of importance. We are not saying their findings were wrong because those patterns are real. But real correlations do not always make good predictors and that is exactly what happened here. Vocabulary complexity is what separates AI writing from human writing when you put it to the test.

Discussion

We went in expecting neural models to win. That is usually how it goes in NLP. They did not, and looking back at why, it makes sense. Neural detectors are trained on one model's output, and they get good at recognizing that specific model. Change the model and the signal disappears. Stylometric features do not have that problem because they are not tied to any one model's quirks.

The Xia et al. situation was worth thinking about. Their correlations were real. We confirmed that. But a feature that correlates with something is not the same as a feature that predicts it well and that gap showed up clearly in the importance of scores.

There are limits to what we can claim here. Our five models represent a snapshot of where things stood at one point in time. LLaMA was already harder to detect than the others and that pattern will likely continue as open-source models get better. The stylometric gap we found may not hold up in two years.

The Mistral test on the live app was a good sign, but it was informal. One model, one test. To really back up the generalization claim you would need a proper held-out evaluation of models that came out after the training data was collected.

Conclusion and Future Work

We had three questions going into this and here is what we found. The first was whether our feature-based models would beat neural models for cross-model detection. They did, and by a lot. Over 81 F1 points were not what we expected. The second was which features would hold up across models we had never seen. We found that short, simple word use was what set human writing apart. syllables_per_word alone gave us over 50% of our importance scores. The voice, tense, and pronoun feature we borrowed from Xia et al. barely showed up, ranking 14th through 40th. They help explain the problem, but our tests show they do not fix it. The third was whether we could get more out of a hybrid ensemble. We did not build one. Our three classical models were already at 99% F1 on their own and mixing in an 18% F1 neural model would have hurt more than it helped.

One thing we did not expect came out of the live app. We ran Mistral text through it, a model we had never trained on, and it came back at 98.7% AI confidence. That told us our features were not just memorizing the five models we used. They picked up something more general than that.

There is still work left to do. The most obvious next step for us is testing on newer models as they come out. We also want to know how well the detector holds up if someone tries to get around it by paraphrasing their text. That is a real weak spot. We would also like to try this in other languages to see if the same word complexity patterns show up outside English. And the perplexity and embedding features we dropped early on are still on our list to revisit when we have more compute to work with.

References

1. Crossley, S. A., Baffour, P., Tian, Y., Franklin, A., Benner, M., & Boser., U. (2024). A large-scale corpus for assessing written argumentation: PERSUADE 2.0. *Assessing Writing*, 61. <https://www.kaggle.com/datasets/nbroad/persaude-corpus-2>
2. Desaire, H., Chua, A. E., Isom, M., Jarosova, R., & Hua, D. (2023). *ChatGPT or academic scientist? Distinguishing authorship with over 99% accuracy using off-the-shelf machine learning tools*. ArXiv.org. <https://arxiv.org/abs/2303.16352>
3. Gehrmann, S., Strobel, H., & Rush, A. M. (2019). GLTR: Statistical Detection and Visualization of Generated Text. *ArXiv:1906.04043 [Cs]*. <https://arxiv.org/abs/1906.04043>
4. He, P., Liu, X., Gao, J., & Chen, W. (2021). DeBERTa: Decoding-enhanced BERT with Disentangled Attention. *ArXiv:2006.03654 [Cs]*, 6. <https://arxiv.org/abs/2006.03654>
5. Li, Y., Li, Q., Cui, L., Bi, W., Wang, L., Yang, L., Shi, S., & Zhang, Y. (2023). Deepfake Text Detection in the Wild. *ArXiv (Cornell University)*, 3. <https://doi.org/10.48550/arxiv.2305.13242>
6. Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L., & Stoyanov, V. (2019, July 26). *RoBERTa: A Robustly Optimized BERT Pretraining Approach*. ArXiv.org. <https://arxiv.org/abs/1907.11692>
7. Mitchell, E., Lee, Y., Khazatsky, A., Manning, C. D., & Finn, C. (2023). DetectGPT: Zero-Shot Machine-Generated Text Detection using Probability Curvature. *ArXiv:2301.11305 [Cs]*, 2. <https://arxiv.org/abs/2301.11305>
8. Wang, Y., Mansurov, J., Ivanov, P., Su, J., Shelmanov, A., Tsvigun, A., Afzal, O. M., Mahmoud, T., Puccetti, G., Arnold, T., Aji, Alham Fikri, Habash, N., Gurevych, I., & Nakov, P. (2024). *M4GT-Bench: Evaluation Benchmark for Black-Box Machine-Generated Text Detection*. ArXiv.org. <https://arxiv.org/abs/2402.11175>
9. Xia, Y., Stańczak, K., & Roth, B. (2026). *Explaining Generalization of AI-Generated Text Detectors Through Linguistic Analysis*. ArXiv.org. <https://arxiv.org/abs/2601.07974>
10. Xu, H., Ren, J., He, P., Zeng, S., Cui, Y., Liu, A., Liu, H., & Tang, J. (2023). *On the Generalization of Training-based ChatGPT Detection Methods*. ArXiv.org. <https://arxiv.org/abs/2310.01307>